

Document: P2034R3
Author: Ryan McDougall <mcdougall.ryan@gmail.com>
Nestor Subiron Montoro <nsubiron@gmail.com>
Audience: EWG-I
Project: ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Partially Mutable Lambda Captures

Or

A More Uniform Const for Lambdas

Revision History

Changes from R2:

- Update author email addresses.
- Rename `any_invocable` to `move_only_function`.

Changes from R1:

- Add discussion of const captures on move construction and assignment.
- Add vocabulary type `as_mutable`.
- Add alternative implementation strategy for const members.
- Selective move feature in top section.

Changes from R0: [Concerns from EWG-I](#)

- Interactions with `this` pointer.
- Interactions with init-capture packs.
- Clarify const as it applies to pointers.
- Add const-reference use case.
- Expanded prose.

Background

Lambdas were introduced in [N2550](#), and while [previous](#) drafts considered mutable capture by value, the original wording left captures entirely const. [N2658](#) salvaged mutable for *all* captures by allowing the `mutable` keyword to modify the call.

[P0288](#) (`move_only_function`) was approved by LEWG, and a central improvement is that it respects the const modifier on function types (ie. `move_only_function<void(int) const>`). This means an `move_only_function` with a const modifier on its call type will only bind to lambdas that are not marked `mutable`.

A type that is “[logically const](#)” is a type that has some mutable members that do not fundamentally change the invariants of the object, even when it is `const`. This means `move_only_function`, and *any* other `const`-correct library, *cannot* work with logically `const` lambdas.

Motivation

Type erased callables like `std::move_only_function` are the backbone of most asynchronous systems. Users of such systems close their operations in lambdas and place them in a concurrent queue to be processed elsewhere. Performance is often key in such systems, and such operations may want its own local reusable scratch memory. Or perhaps an accumulator for hysteresis over multiple calls.

```
struct MyRealtimeHandler {
    Callback callback_;
    State state_;
    mutable Buffer accumulator_;

    void operator()(Timestamp t) const {
        callback_(state_, accumulator_, t);
    }
};
```

```
concurrent::queue<move_only_function<void(Timestamp) const> queue;
queue.push(MyRealtimeHandler{f, s});
```

Moreover, a classic use for mutable members in bespoke classes is `std::mutex`.

```
struct MyThreadedAnalyzer {
    State state_;
    mutable std::mutex mtx_;

    void operator()(Slice slice) const {
        std::lock_guard<std::mutex> lock{mtx_};
        analyze(state_, slice);
    }
};
```

```
concurrent::queue<move_only_function<void(Slice) const> queue;
queue.push(MyThreadedAnalyzer{s});
```

Lambdas in such cases require work-arounds, such as abandoning logical `const` correctness, or using intermediary types (such as `std::ref`) that do not propagate constness.

Proposal

Mutable Capture By Value

Allow [lambda capture initialization](#) to be `mutable` qualified, as below. This would have the effect of declaring the captured variable to be mutable.

```
auto a = [mutable x, y]() {};
```

```
// equivalent to:
```

```
struct A {  
    mutable X x;  
    Y y;  
    void operator()() const {}  
};
```

Before	After
<pre>struct A { const State state; mutable Buffer buf; void operator()() const { // ... } }; // manual bespoke type move_only_function<void() const> f = A{s, b};</pre>	<pre>move_only_function<void() const> f = [s, mutable b] { // ... };</pre>
<pre>template <typename T> class as_mutable { mutable T value; public: T& ref() const { return value; } }; // one-off vocabulary type move_only_function<void()> f = [s, b = as_mutable<Buffer>{}]() { auto& buffer = b.ref(); // ... };</pre>	<pre>move_only_function<void() const> f = [s, mutable b] { // ... };</pre>
<pre>// loss of const correctness move_only_function<void()> f = [s, b]() mutable { // ...</pre>	<pre>move_only_function<void() const> f = [s, mutable b] { // ...</pre>

<code>};</code>	<code>};</code>
<code>// loss of regular value type</code> <code>move_only_function<void()> f =</code> <code> [s, buf_ptr = &b]() mutable {</code> <code> // ...</code> <code> };</code>	<code>move_only_function<void() const> f =</code> <code> [s, mutable buf = b] {</code> <code> // ...</code> <code> };</code>

Selective Moves with init-capture Packs

Following the direction set out in [P2095](#), using the example in [P0780](#), we are able to move arguments from caller, to lambda, to callee -- without having to stop at the lambda:

```
template <class... Args>
auto delay_invoke_foo(Args... args, State s) {
    return [s, mutable ...args=std::move(args)] { // <-- new
        return foo(s, std::move(args)...);        // <-- improved
    };
}
```

Possible Extensions

Note: these are proposed as optional, and have demonstrated some user interest.

Extensions are motivated by use cases, and listed in order of perceived usefulness -- however it should be noted that they also introduce increasing consistency and symmetry, which the authors believe is a justification in its own right.

1. Const Capture on Mutable Call Operator

If lambda capture initialization can be modified by `mutable` and lambda closure call can be modified by `mutable`, then lambda closure calls modified by `mutable` should be able to declare some of their captures `const` – an inversion of this paper’s core proposal.

Value

If most of the values captured are mutable, but one should be `const`, then this variation would be shorter and more readable.

Implementation

```
auto b = [x, const y]() mutable {};
```

`// equivalent to:`

```
struct B {
```

```

X x;
const Y y;
void operator() () {}
};

```

A `const` member would make the lambda closure assignment operators deleted, but lambda closures with captures [already delete the copy assignment operator](#), and arguably closure assignment is lesser used.

A `const` member would also cause the move constructor to be implemented via copy, potentially causing it non-noexcept, depending on the copy constructor of the `const` member.

We can avoid these problems with another implementation strategy:

// **equivalent to:**

```

struct B {
    X x;
    Y _y;
    void operator() () {
        const Y& y = _y;
    }
};

```

However this has the wrinkle that `decltype(y)` would be visibly different from what might be expected.

We could also invoke compiler magic using “as-if”

// **equivalent to:**

```

struct B {
    X x;
    Y y;
    void operator() () {
        // as-if y was declared const
    }
};

```

Before	After
<pre> struct A { const float *iter; const float *const end; void operator() () { for(; iter != end; ++iter) { // end never modified... </pre>	<pre> move_only_function<void()> f = [iter = a.cbegin(), const end = a.cend()] () mutable { for(; iter != end; ++iter) { // end never modified... </pre>

<pre> } } }; // manual bespoke type move_only_function<void()> f = A{ a.cbegin(), a.cend() }; </pre>	<pre> } }; </pre>
<pre> // extraneous mutable copy move_only_function<void()> f = [iter = a.cbegin(), end = a.cend()] () { auto copy = iter; for(; copy != end; ++copy) { // end never modified... } }; </pre>	<pre> move_only_function<void()> f = [iter = a.cbegin(), const end = a.cend()] () mutable { for(; iter != end; ++iter) { // end never modified... } }; </pre>

2. Const Capture by Reference

Capture by reference is not implicitly `const`, as capture by value is. However there are situations where it would be useful to capture by const reference, such as when a read-only object is too large to copy, or as a novel means to create a read-only code block.

Value

Enforce const on objects captured reference.

Implementation

```
auto b = [&x, const &y]() {};
```

// **equivalent to:**

```

struct B {
    X &x;
    const Y &y;
    void operator()() const {}
};

```

We could also invoke compiler magic using “as-if”

// **equivalent to:**

```

struct B {
    X &x;
    Y &y;
    void operator()() {

```

```

        // as-if y was declared const Y&
    }
};

```

Before	After
<pre> struct A { const Huge &huge; void operator() () const { // huge.mutate(); is error } }; // manual bespoke type move_only_function<void() const> f = A{huge}; </pre>	<pre> move_only_function<void() const> f = [const &huge] { // huge.mutate(); is error }; </pre>
<pre> // extraneous cast move_only_function<void() const> f = [&huge = static_cast<const Huge&>(huge)] { // huge.mutate(); is error }; </pre>	<pre> move_only_function<void() const> f = [const &huge] { // huge.mutate(); is error }; </pre>
<pre> X a, b, c; a = foo(); b = bar(); c = baz(); { // manual redeclaration and assignment const X& const_a = a; const X& const_b = b; const X& const_c = c; // ... enter const context } </pre>	<pre> X a, b, c; a = foo(); b = bar(); c = baz(); [const &] { // ... const context }(); </pre>

3. Const Call Operator

For symmetry with the call operator of bespoke types, declaring the lambda const should not be an error.

```
auto c = [x]() const {};
```

// equivalent to:

```

struct C {
    X x;
    void operator() () const {}
};

```

4. Const Capture on Const Call Operator

For symmetry and principle of least surprise, declaring a const capture of a const lambda should not be an error.

```
auto c = [const x]() {};
```

See Const Capture on Mutable Call Operator.

5. Mutable Capture on Mutable Call Operator

For symmetry and principle of least surprise, declaring a mutable capture of a mutable lambda should not be an error.

```
auto c = [mutable x]() mutable {};
```

```
// equivalent to:
```

```
struct C {  
    mutable X x;  
    void operator()() {}  
};
```

Benefits of Consistency and Symmetry

The core benefits of extensions 3, 4 and 5 is lower cognitive load for programmers learning C++, and principle of least surprise. We can teach why lambdas default the way they do, but lambdas should have consistent and symmetric vocabulary for teaching how lambdas transform into callable types under the hood.

Experienced users will also benefit from additional self-documentation, especially in critical reliability contexts where verbosity and redundancy are preferred. Users would declare the lambda `mutable` or `const` according to ideal or majority semantics, and some minority of capture initialization would be the opposite, as an exception.

Concerns

1. Move construction with const captures

Const members cannot be moved from effectively, and lambdas with const captures would silently inhibit the (potentially noexcept) move constructors of what it captures by value, in favor of their copy constructors. This is more than just a pessimization, it may turn a non-throwing move operation into a potentially throwing copy.

```

struct A {
    std::string s;
};
static_assert(std::is_move_constructible_v<A>);
static_assert(std::is_nothrow_move_constructible_v<A>);

struct B {
    const std::string s;
};
static_assert(std::is_move_constructible_v<B>);
static_assert(!std::is_nothrow_move_constructible_v<B>);

```

While users can do this today with classes, the concern is this would be making it easier to do unwittingly via lambdas.

Whether the improved symmetry and teachability of const capture lambdas is worth the possible footgun, remains an open question.

2. Assignment operations with const captures

Const members cannot be assigned to, and lambdas with const captures would be creatable but not assignable. In practise it is rare to assign a lambda, and wrappers like `std::function` use assignment to replace the object.

3. East v. West Const

In both East or West-const, the const always appears before the identifier. This proposal does not change that.

4. Pointer to Const v. Const Pointer

Current lambda behavior mandates bitwise const, which is const-pointer (not pointer to const). This proposal seeks to continue and not to modify that rule.

```

auto c = [const x = ptr]() {
    *x = {}; // ok
    x = nullptr; // error
};

```

5. Interactions with `this`

The keyword `this` is a prvalue expression, and is special cased with regard to lambda captures. As such, the meaning of `mutable this` and `const this` doesn't have obvious semantics -- or if we defined them may be hard to teach. We recommend these two combinations be disallowed until further experience is accrued.

Students will likely expect the following to compile (it would not):

```

struct A {
    void mutate() {}
    void test() const {
        [mutable this] {
            this->mutate();
        }();
    }
};

```

Whereas the following would compile and work:

```

struct B {
    void mutate() {}
};

void test(B* that) {
    [mutable that] {
        that->mutate();
        that = nullptr;
    }();
}

```

Recall const pointer lambda capture is *bitwise* const, which affects if the pointer itself can be modified. The `this` pointer can never be modified and so `mutable this` or `const this` would either be meaningless if bitwise const, or inconsistent if logically const.

The meaning of `mutable *this` and `const *this` is much clearer, but for the sake of consistency when teaching “`this` is special”, we recommend dis-allowing this form as well.

6. These extensions seem like a lot. Could traps be lurking?

Everything being proposed has a direct and consistent transformation into callable types that are *already allowed*. Consistency and symmetry improve the teachability of lambdas, and the defaults chosen for C++11 lambdas can be easily explained.

That said, this proposal is easily separable.

Thanks

Thanks Patrick McMichael for suggesting the idea. Thanks to Nevin Liber, Matt Calabrese for offering important corrections. Thanks to Nevin Liber, Davis Herring, and Barry Revzin for examples and suggestions. Thanks to Nestor Subiron Montoro for becoming second author.